

EXERCISE PART 1 : Knowing where to start

2.1 The project is organised into different use cases and coding exercises. The use - cases folder contains different categories of tasks, such as algorithms, java, javascript and practical projects like a task manager. Each category contains exercises that focus on a particular programming language or problem solving concept. The README.md provides information about the project, while Git files such as .gitignore and .git support version control.

2.2 TECHNOLOGIES :

\- Java : There's a Java folder and therefore Java exercises.

\- JavaScript : There's a JavaScript folder / tasks.

\- Node.js : It is likely used for running the JavaScript exercises locally, especially since i've been using Node to run my JavaScript code.

\- Git : The .Git folder shows the project is using Git for version control.

\- Github : Since the project is hosted / managed through Github.

\- Markdown : The README.md file uses markdown for documentation.

FRAMEWORKS :

\- No specific framework has been identified yet from the files i have reviewed.

2.3 The main components appear to be the use cases, code algorithms, Java exercises, JavaScript exercises and the task manager project. The README file also serves as the main documentation for the project.

3.2 Initial Understanding:

\- My understanding is there are a list of both files and folders some of which are coding exercises.

\- There are different programming languages outlined by the files meaning the exercises can be done in separate programming languages.

Questions:

\- What is the overall purpose of the project?

\- What are the main components of the project and what is their purpose?

AI ANALYSIS:

\- What the application does : It seems to be a collection of AI / code exercises and practical programming tasks, rather than one single application. The presence of use cases, algorithms, Java, JavaScript and a task manager suggests the repository is organised around different coding exercises.

\- Technology Stack : It appears to use Java, JavaScript, Node.js, Git, Github and markdown. No specific framework has been identified yet.

\- Folder structure : The folder structure seems to follow a category - based organisation, where the use - cases folder contains different types of programming exercises and projects, such as algorithms, Java, Javascript and the task manager.

\- Main folders :

- 1\. Use cases/ : contains the different coding exercises.
- 2\. code algorithms/ : likely contains algorithm and problem solving exercises.
- 3\. java/ : contains Java based exercises.
4. Javascript : contains Javascript exercises.
- 5.Task manager : appears to be a practical application / project involving task management.
- 6\. README.md : project documentation and instructions.
- 7..gitignore : specifies files Git should ignore.
- 8..git/ : contains Git's version control information.

4.1 Misconceptions :

- 1\. Thinking the project is one single application - It looks more like a collection of coding exercises and smaller projects / use cases.
- 2\. Thinking every folder is a framework - Folders such as Java, JavaScript and code - algorithms are categories or technologies, not necessarily frameworks.
- 3\. Assuming JavaScript requires package.json - package.json is common in Node.js projects, but a JavaScript exercise doesn't necessarily need one.
- 4\. Assuming there must be a framework - We haven't seen evidence of frameworks like React, Express, or Spring Boot yet.

4.2 Important Entry points :

\- The main starting point for understanding the repository is README.md, while the actual execution entry points are located within individual exercises in the use - cases folder. For Java exercises, this may be a class containing a main method, while JavaScript exercises may be executed from their relevant .js files.

Architectural patterns :

\- The project mainly follows a use - case based and modular structure. Exercises are grouped by topics and programming languages rather than being organised as one large application. This makes individual exercises relatively independent and allows multiple programming languages and technologies to be explored within the same repository.

4.3 - The key components of the project include the README.md, use - cases folder, individual programming exercises, .gitignore, and .git folder. The README provides project documentation, while the use - cases folder organises the different exercises and topics. The programming files contain the solutions or code for individual exercises. The .gitignore controls which files Git ignores, and the .git folder manages version - control information. GitHub provides the platform for storing and managing the repository.

Exercise Part 2 :

1.1 - I did not identify any specific export functionality in the codebase. The project appears to focus on programming exercises rather than exporting application data into files such as CSV, PDF or Excel. The project mainly consists of programming exercises and documentation, so there does not appear to be a dedicated file - export feature.

1.2 - I searched the codebase for existing data transformation and external file operations. I found no existing functionality specifically for exporting data to

external files such as CSV. The project mainly contains programming exercises, so the new Task Export to CSV feature would need to introduce the required CSV formatting and file - writing functionality.

1.3 - I searched the codebase for reusable utility functions related to file operations. I did not identify any existing helper functions for creating, reading, writing or exporting files. Therefore, the CSV export feature would likely need to implement its own file handling functionality.

2.1 Hypothesis - I think the task data export functionality would belong within the use - cases folder, alongside the task - related exercise or code. If the project later introduces a dedicated utility or file - handling folder, the CSV formatting and file - writing logic could be placed there so it can be reused. The export functionality should be connected to the task data rather than being placed in the main documentation files.

2.2 - Existing components that might need to be modified include the task - related code within the use - cases folder, because it would need to provide the task data to the export functionality. Any existing task - management or data - formatting functions may also need to be modified to prepare the data in CSV format. If there is an existing interface or command used to interact with tasks, it may also need to be updated to include an export option.

2.3 Search terms used:

\- task

\- export

\- CSV

\- file
\- save
\- write
\- read
\- format
\- convert

Files/folders found:

README.md - project documentation and instructions.

.gitignore - Git ignore configuration.

use - cases/ - contains the project's topics, subtopics and exercises.

3.2 1. My approach to the feature :

* Find the tasks - Turn the tasks into CSV format - Create / save the CSV file

What to search for :

* Where the tasks are handled.

- * Where data / file operations happen.
- * Whether there is already something related to CSV / exporting.

2\. Look at the project structure first :

- * README.md
- * .git
- * .gitignore
- * use - cases
- * java
- * javascript task manager
- * code algorithms

3\. Search for obvious words :

- * task
- * export
- * csv
- * file
- * download

4\. Follow where the tasks are handled :

- * Task manager.js may be relevant because it appears to be responsible for managing the task data that would eventually need to be exported.

5\. Look for file handling :

* I searched the project folders for existing helper or utility functions that could be reused for file operations. I did not find any obvious helpers related to CSV export or file handling.

6\. Look for helpers :

* I could not find any existing helpers for file operations or CSV exporting. I would investigate the task management code to see whether any existing functions could be reused.

7\. Build my hypothesis :

* Based on my initial search, i think the Task Export to CSV functionality would most likely belong near the existing task management code. The task manager appears to be responsible for accessing and managing the task data, while existing utility or file - handling functions could potentially be reused to create the CSV file. I would investigate these areas further before deciding exactly where the new functionality should be added.

8\. Tracing the feature :

* I would trace the feature by following where the task data is created, stored and used. I would search for relevant words such as task, add, edit, delete and export to see which files are connected to the feature. If i cannot find a specific export function, i would look at the existing task - related code to understand where the new feature could fit.

9\. Check whether something similar exists :

* I searched the codebase for similar features, such as exporting, downloading or saving task data. I could not find an existing feature that exports tasks to a CSV file.

10\. Check if tests exist :

* I found a test folder in the codebase. I checked it to see if there were any tests related to task exporting or CSV files, but i did not find any.

3.3 - I looked through the repository structure to find similar features or patterns. The project is organised into different use - case folders, including task - manager, testing - 001, refactor - functions and refactor - patterns. These folders may contain examples of how similar functionality is structured or tested. I also checked the README.md for information about the project. I did not find an exact matching feature from the folder names alone, so I would need to explore the relevant folders further to confirm which patterns can be reused.

4.1 - I would implement the Task Export to CSV feature in the task-manager section of the codebase, because this is where the task-related functionality is located. I would first look for the files that handle task data and then add the export functionality there. I would also check whether there are any existing utility or file-handling functions that could be reused.

4.2 - The main component that would be affected is the task-manager, because it contains the task-related functionality. The task data would need to be accessed by the new CSV export feature. If there are any file-handling or utility components in the project, they may also need to be updated to support creating and saving the CSV file.

4.3 - 1. Go to the use-cases folder and locate the task-manager section where the task-related code is stored.

2\|. Examine the existing task code to understand how tasks are created, stored, and displayed.

3\|. Identify the part of the task-manager code where the export functionality could be added without changing the existing task features.

4\|. Add a function that takes the existing task information and converts it into CSV format.

5\|. Add the file-handling code needed to create and save the CSV file.

6\|. Test the export feature using the existing task data and check that the CSV file contains the correct information.

7\|. Make sure the new feature does not interfere with the existing functionality of the task-manager.

Exercise Part 3

1.1 The core domain entities are defined in models.js:

\- Task - represents an individual task and its information.

\- TaskPriority - represents the priority level assigned to a task.

\- TaskStatus - represents the current status or state of a task.

These three entities form the main domain model of the Task Manager and are exported from `models.js` for use by other parts of the application.

1.2 The business logic related to tasks is mainly found in `models.js` within the `Task` class. The main logic includes:

`\- update()` - updates the task's existing properties and records when the task was updated.

`\- markAsDone()` - changes the task status to `DONE`, records the completion time, and updates the timestamp.

`\- isOverdue()` - checks whether a task has passed its due date and is not yet completed.

`\- The Task constructor also sets default values, such as the initial status being TODO and the default priority being MEDIUM.`

These methods contain rules that control how tasks behave in the Task Manager.

1.3 - Some terminology specific to this application includes `TaskStatus`, which describes the stage of a task, and `TaskPriority`, which describes how important a task is. The application also uses statuses such as `TODO`, `IN\_PROGRESS`, `REVIEW`, and `DONE`, and priority levels such as `LOW`, `MEDIUM`, `HIGH`, and `URGENT`. Other task-specific concepts include due dates, tags, `completedAt`, and `updatedAt`.

2.1

ENTITY :	WHAT IT REPRESENTS :	RELATIONSHIPS TO TASK :
Task	The main task being managed	Main entity
TaskStatus	The current stage of a task	Task has a status

|TaskPriority|How important a task is|Task has a priority|
| | | |

2.3 – One thing I found confusing was how the different task statuses are supposed to be used. For example, I can see statuses such as TODO, IN_PROGRESS, REVIEW, and DONE, but it is not completely clear what causes a task to move from one status to another.

I was also unsure about when `completedAt` is updated when a task is marked as done. I would need to check the rest of the code to understand how this is handled.

Another question I have is whether there are any rules about changing the task priority or due date after a task has been created.

3.2 – My current understanding is that the application is a Task Manager. The `Task` class is the main entity and contains information such as the task title, description, priority, status, due date, and tags. `TaskStatus` shows the stage of a task, while `TaskPriority` shows how important the task is.

I understand that the `update()` method is used to change information about a task and that `markAsDone()` changes the task's status to DONE.

My questions are:

* What rules determine when a task should change from one status to another?

* When a task is marked as done, when is `completedAt` updated?

* Are there any rules about changing a task's priority or due date after it has been created?

3.3 – I asked the AI to test my understanding of the domain model by asking questions about the business logic. The questions included:

- 1\. What is the main purpose of the Task class?
- 2\. What is the difference between TaskStatus and TaskPriority?
- 3\. What happens when a task is marked as done?
- 4\. Why does a task have a due date and a completed date?
- 5\. What rules might determine when a task changes from one status to another?

4.1 1. What is the main purpose of the Task class?

The Task class represents a task in the Task Manager. It stores information such as the title, description, priority, status, due date, tags, and dates related to the task.

2\. What is the difference between TaskStatus and TaskPriority?

TaskStatus shows the current stage of a task, such as TODO, IN_PROGRESS, REVIEW, or DONE. TaskPriority shows how important the task is, ranging from LOW to URGENT.

3\. What happens when a task is marked as done?

When a task is marked as done, its status is changed to DONE using the `markAsDone()` method. The `completedAt` field is also intended to record when the task was completed, although the full method would need to be checked to confirm exactly how it is updated.

4\.. Why does a task have a due date and a completed date?

The due date shows when the task is expected to be completed. The completed date records when the task was actually completed. This allows the application to keep track of the task's deadline and completion.

5\.. What rules might determine when a task changes from one status to another?

The code shows the available statuses, but it does not show all the rules for changing between them. For example, a task could move from TODO to IN_PROGRESS when work starts and eventually to DONE when it is completed. The exact rules would need to be checked in the other parts of the application.

4.2

ENTITY	WHAT IT REPRESENTS	RELATIONSHIP TO TASK
Task	The main item being managed	Main entity
TaskStatus	The current stage of the task	A task has a status
TaskPriority	How important the task is	A task has a priority
Tags	Labels used to organise task	A task can have tags
Due Date	The date the task is expected to be completed	A task can have a due date.
Completed Date	The date the task was completed	Records when a task is completed
Created / Updated Dates	Tracks when the task was created and last changed	Keeps track of task changes

4.3

DOMAIN TERM	MEANING
- -	
Task	An item of work that needs to be completed
Task Status	Shows the current stage of a task
TODO	A task that has not been started
IN_PROGRESS	A task that is currently being worked on
REVIEW	A task that needs to be checked or reviewed
DONE	A task that has been completed
Task Priority	Shows how important or urgent a task is
LOW	A task with low importance
MEDIUM	A task with normal importance
HIGH	A task with high importance
URGENT	A task that needs immediate attention
Due Date	The date by which a task should be completed
Completed At	Records when a task was completed
Tags	Labels used to organise or categorize tasks
Created At	Records when a task was created
Updated At	Records when a task was last changed

Exercise Part 4

1. Scenario :

The new rule is:

Tasks that are overdue for more than 7 days should automatically be marked as abandoned unless they are marked as high priority.

2\. Planning :

Files I would need to modify

\- models.js - I would modify this file because it contains Task, TaskStatus, and TaskPriority. I would add the new ABANDONED status and the logic for checking whether a task should be abandoned.

\- app.js - I would check and possibly modify this file so that the overdue check is performed when the application runs.

\- storage.js - I would check whether this file needs changes so that the new ABANDONED status is saved correctly.

\- cli.js - I would check whether the CLI displays task statuses and whether it needs to be updated to show ABANDONED.

Changes I would make

1\. Add ABANDONED to the TaskStatus values.

2\. Check whether a task has a due date.

3\. Check whether the due date is more than 7 days in the past.

4\. Check the task's priority.

5\. If the task is more than 7 days overdue and is not high priority, change its status to ABANDONED.

6\. Make sure high-priority tasks are not automatically marked as abandoned.

7\. Make sure the new status is saved and displayed correctly.

Questions I would ask the team

\- Should urgent tasks also be protected from being marked as abandoned, or only high-priority tasks?

\- Should a task only be abandoned if its status is still TODO, or should tasks in other statuses also be affected?

\- When exactly should the overdue check run?

\- Should the user be notified when a task is automatically marked as abandoned?

\- Should an abandoned task be allowed to be reopened later?

3\. Reflection :

The AI prompts helped me understand that `models.js` is an important part of the implementation because it contains the `Task`, `TaskStatus`, and `TaskPriority` concepts. They also helped me connect the business rule to the existing task properties, such as `dueDate`, `priority`, and `status`.

I am still unsure about exactly where the application checks tasks regularly and how changes to a task are saved. I would need to understand the roles of `app.js` and `storage.js` better before implementing the rule.

My next steps would be to read through `app.js` and `storage.js`, look for existing date or status-checking logic, and check the tests to see how task behaviour is currently tested. I would then discuss the questions above with my team before making changes.

Final Discussion and Reflection:

1.1 - My Approach to Understanding the Codebase

I started by looking at the project structure and identifying the main files and folders. I then looked at the README.md to understand what the project was about and identified the JavaScript Task Manager files, including app.js, cli.js, models.js, and storage.js.

After that, I focused on models.js to understand the domain model. I identified Task as the main entity and looked at TaskStatus and TaskPriority to understand how tasks are organized. I also looked at the task properties and methods such as update() and markAsDone() to understand some of the business logic.

I then created a glossary and a simple table to organize the domain terms and their meanings. I also considered how the different concepts relate to each other.

Finally, I applied my understanding to a new business rule about overdue tasks. This helped me identify which files might need to be changed and what questions I would need to ask the team before implementing the rule.

Using AI questions and explanations helped me break down the codebase into smaller parts and understand the business meaning behind the code rather than only looking at the syntax.

1.2 - One challenge I encountered was understanding the project structure and knowing which files were important. At first, I was not sure what each file was responsible for. I overcame this by looking at the file names, reading the README.md, and examining the code in the different files.

Another challenge was understanding the business logic in models.js. I could see the code, but I was not always sure what concepts such as task status, priority, and completed date meant in terms of how the application works. I overcame this by

breaking the code into smaller parts and using AI to explain the concepts in simpler terms.

I also found it difficult to understand how the different parts of the application are connected. Creating a table showing the relationships between Task, TaskStatus, and TaskPriority helped me organize my understanding.

Overall, I overcame these challenges by exploring the code step by step, asking questions when I was confused, and using diagrams and tables to make the information easier to understand.

1.3 - I used several strategies to understand the codebase. First, I explored the project structure and file names to get an idea of what each part of the application does. This was useful for getting an overall understanding of the project.

I then read the code in models.js to identify the main entities and business logic. This gave me a better understanding of how tasks, statuses, and priorities work.

I also used tables and simple diagrams to organize the relationships between the different concepts. The table was useful for recording detailed information, while the diagram made the relationships easier to see.

Finally, I used AI to ask questions and explain parts of the code that I found confusing. This helped me understand the business meaning of the code rather than only focusing on the programming syntax.

Overall, each strategy helped in a different way: exploring helped me understand the structure, reading the code helped me understand the logic, tables and diagrams helped me organize the information, and AI helped clarify areas I did not understand.

2.1 - The prompt that was most helpful to me was the one where I asked the AI to validate my understanding of the domain model and ask me questions about the business logic.

It helped me understand that Task is the main entity and that TaskStatus and TaskPriority describe the task. The questions also made me think about how the different parts of the application work together instead of just looking at the code.

This prompt was helpful because it allowed me to identify things I understood and areas where I was still confused. It made it easier for me to connect the code to real features in the Task Manager.

2.2 - Next time I approach an unfamiliar codebase, I would start by making a clear plan of the files and folders I need to investigate. I would first read the README.md, then identify the main files and understand what each one is responsible for.

I would also spend more time tracing how the different parts of the application connect before making assumptions. I would ask questions earlier when I do not understand something and use simple tables or diagrams to organize my findings.

This would help me understand the codebase faster and make it easier to identify where new features or business rules should be implemented.

2.3 Additional tools and resources that would complement the AI prompting approach include:

\-README.md - to understand the purpose and basic structure of the project.

\-GitHub - to view the repository, files, commits, and project history.

\-Code editor – such as VS Code, to read and navigate through the code more easily.

\-Search function – to quickly find where specific terms, methods, or classes are used in the codebase.

\-Tests – to understand how existing features are expected to work and to check whether new changes work correctly.

\-Project documentation – to understand technologies, rules, and features that may not be clear from the code itself.

These resources would work together with AI prompting by allowing me to first investigate the actual code and then use AI to explain areas that I do not understand.

Submission :

Task Manager Codebase – Final Summary

1\. Initial vs. Final Understanding

Initial Understanding

At the beginning, I was not completely sure how the Task Manager codebase was organized or what each file was responsible for. I knew that it was a Task Manager application, but I needed to investigate the project structure and the JavaScript files to understand how the application worked.

I identified the main JavaScript files as `app.js`, `cli.js`, `models.js`, and `storage.js`. At first, I was unsure how these files were connected and where the main business logic was located.

Final Understanding

After exploring the codebase, I understood that `models.js` contains the main domain model. The `Task` class is the main entity and contains information such as the task title, description, priority, status, due date, tags, and dates.

I also understood that `TaskStatus` describes the stage of a task, with values such as `TODO`, `IN_PROGRESS`, `REVIEW`, and `DONE`. `TaskPriority` describes how important a task is, with values from `LOW` to `URGENT`.

I learned that methods such as `update()` and `markAsDone()` contain business logic because they control how information about a task changes.

2\. Most Valuable Insights Gained From the Prompts

The prompts helped me understand the codebase step by step.

The prompt about identifying the core entities helped me recognize that `Task` is the main entity and that `TaskStatus` and `TaskPriority` describe different aspects of a task.

The prompt about looking for business logic helped me understand that business logic is not only about large features. Methods such as `update()` and `markAsDone()` can also contain rules about how tasks behave.

The prompt about terminology and concepts helped me understand terms such as status, priority, due date, completed date, and tags in terms of what they mean to the user.

Creating a table and entity diagram also helped me visualize how the different concepts are connected.

Finally, asking AI to test my understanding with questions helped me identify areas where I was still unsure, such as when a task should change status and how `completedAt` is updated.

3\ Approach to Implementing the New Business Rule

The new business rule is:

Tasks that are overdue for more than 7 days should automatically be marked as abandoned unless they are marked as high priority.

Based on my understanding, I would first add an `ABANDONED` status to `TaskStatus`. I would then check the task's due date to determine whether it has been overdue for more than seven days.

The task's priority would also need to be checked. If the task is more than seven days overdue and is not high priority, its status should be changed to `ABANDONED`. High-priority tasks should not be automatically abandoned.

I would investigate `models.js` first because it contains the task model and status values. I would then check `app.js` to find where task-related operations are performed and `storage.js` to understand how task changes are saved. I would also check `cli.js` to see whether the new status needs to be displayed to the user.

Before implementing the rule, I would ask my team questions such as whether urgent tasks should also be protected, which task statuses should be affected, when the overdue check should run, and whether abandoned tasks can be reopened.

4\. Strategies for Approaching an Unfamiliar Codebase

I have developed several strategies that I can use when working with an unfamiliar codebase in the future.

First, I would start by reading the README.md and looking at the project structure. I would identify the main files and folders before trying to understand individual pieces of code.

Next, I would identify the main entities and follow how they are used throughout the application. I would search for important classes, methods, and keywords to see where particular features are implemented.

I would also use simple tables and diagrams to organize the relationships between different parts of the application. If I find something confusing, I would ask specific questions instead of making assumptions.

I would check existing tests because they can show how the application is expected to behave. I would also use GitHub, project documentation, and my code editor alongside AI prompting.

Overall, this exercise taught me that understanding an unfamiliar codebase is easier when I break it into smaller parts, understand the domain concepts first, and then trace how those concepts are used in the application.